Generative AI-Based Red-Teaming for Autonomous UAV Navigation

CS787 Course Project

Snehil Chatterjee (241026) · snehil24@iitk.ac.in

Code, results and demo video:
https://github.com/Snehil06/cs787-uav-redteaming

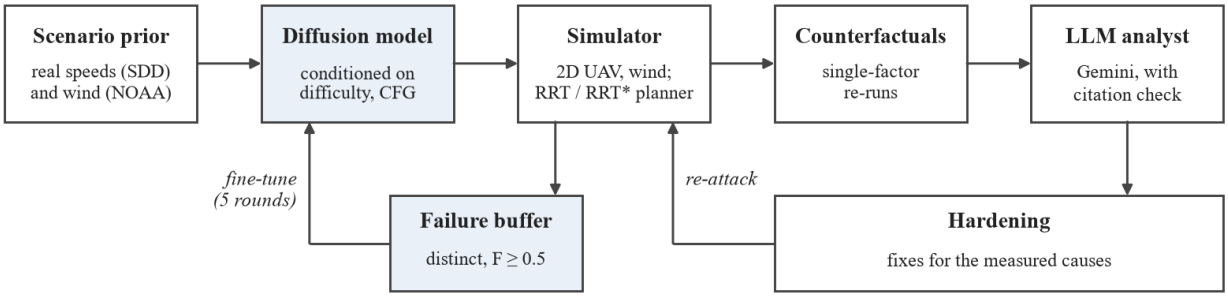


Figure 1: Overview. A conditional diffusion model, trained on scenarios whose obstacle speeds and winds follow real data, generates scenarios for an unmodified RRT/RRT* UAV planner. Simulated failures fill a de-duplicated buffer that fine-tunes the generator each round. Every failure is re-flown under single-factor interventions to measure its causes, which a grounded LLM turns into explanations and which guide targeted hardening; the hardened system is then attacked again. Shaded blocks form the closed red-teaming loop (Algorithm 1).

Abstract

Sampling-based planners such as RRT are widely used for UAV navigation, yet their failures under wind, delayed sensing and moving obstacles are rare and hard to find by random testing. Following DiffScene, which showed that diffusion models generate realistic safety-critical driving scenarios, we build a red-teaming framework for UAV path planning that *generates*, *explains* and *fixes* navigation failures. A conditional diffusion model over a 46-dimensional scenario space, trained on scenarios grounded in the Stanford Drone Dataset and NOAA wind records, is steered toward difficulty with classifier-free guidance and fine-tuned in closed loop on the failures it finds. Each failure is re-flown under single-factor interventions to measure its causes, and an LLM (Gemini) explains it under a citation check. With 2,000 simulated flights per run, our method finds 552 distinct failures versus 510 for CMA-ES and 214 for random sampling, spread over 3.5× more of the scenario space than CMA-ES. Diffusion is far more valid and realistic than a tuned conditional VAE (87% vs. 50% valid). Measured counterfactuals raise the LLM's recall of true causes from 34% to 96%, whereas retrieving similar past cases does not help. Fixes derived from the measured causes survive 79% of past failures and cut the failure rate from 12.0% to 4.7%; re-attacking the hardened system, our red team still finds 2.1× more failures than random sampling.

1 Introduction

Autonomous UAVs rely on path planners that must keep working when the world is not what the planner expects: wind pushes the vehicle off its path, obstacles appear late or move, and sensing is limited, noisy and delayed. Such failures are rare: on random scenarios our test planner fails about 11% of the time, and the most informative failures arise from *combinations* of factors that random sampling seldom produces. Black-box search finds failures faster but tends to collapse onto one failure mode (Corso et al. 2021). Testing therefore needs a generator that is both *targeted* and *diverse*, and that stays within physically plausible scenarios.

DiffScene (Xu et al. 2023; 2025) addressed the same tension for autonomous driving by using a diffusion model as a realistic prior over scenarios and guiding it toward safety-critical events. It argued that diffusion represents the low-density tail of a scenario distribution better than VAE-based generators such as STRIVE (Rempe et al. 2022) and stays more realistic than adversarial optimisation such as AdvSim (Wang et al. 2021). We take this idea to UAV navigation and extend it in three directions.

Contributions.

- **Closed-loop diffusion red-teaming** for an unmodified open-source RRT/RRT* stack: a conditional DDPM with classifier-free guidance, fine-tuned round by round on simulator-confirmed failures, instead of gradient guidance through a learned surrogate.
- **Real-data grounding and a fairness rule.** Obstacle speeds come from the Stanford Drone Dataset and wind from NOAA records; scenarios in which an obstacle

appears inside the UAV's reaction distance are rejected, so reported failures are avoidable in principle.

- **Measured causes and a grounded LLM analyst.** Six single-factor counterfactual re-runs label every failure with its causes; an LLM explains failures under a script-checked citation rule, and an ablation measures which evidence makes its explanations faithful.
- **Closing the loop.** The measured causes drive four targeted fixes; we measure regression, side effects and a re-attack of the hardened system.

2 Related Work

Safety-critical scenario generation. A survey of black-box safety validation (Corso et al. 2021) distinguishes sampling, optimisation and learned generators. AdvSim (Wang et al. 2021) perturbs actor trajectories adversarially; STRIVE (Rempe et al. 2022) optimises in the latent space of a learned VAE traffic model. DiffScene (Xu et al. 2023; 2025) replaces the VAE prior with a diffusion model and guides each denoising step with safety, functionality and constraint objectives, reporting more realistic and more transferable scenarios than six baselines. Our work follows DiffScene's prior-plus-guidance design but targets UAV planners, uses classifier-free guidance (Ho and Salimans 2021) with closed-loop fine-tuning, and adds causal analysis.

Generative models. We use DDPMs (Ho, Jain, and Abbeel 2020) with a cosine schedule and FiLM conditioning, compared against a conditional VAE (Sohn, Lee, and Yan 2015). Our search baseline is CMA-ES (Hansen 2016).

Planners and explanation. RRT (LaValle 1998) and RRT* (Karaman and Frazzoli 2011) are taken unmodified from PythonRobotics (Sakai et al. 2018). Causes are defined by intervention. Our LLM analyst uses Gemini with retrieval of similar past failures (retrieval-augmented generation); we test whether retrieval or measured interventions make explanations faithful.

3 Problem Formulation

A scenario is a vector x in $\mathbf{R}^{46}$: start, goal and cruise speed of the UAV; five obstacle slots (position, velocity, radius, appearance time, active flag); mean wind, gust amplitude and onset; sensing range and position-noise level. A rule set V(x) rejects physically impossible scenarios (overlapping start and obstacles, out-of-range values, too-short missions) and, new in this work, unfair ones (Sec. 4.1). The simulator f flies the planner through x and returns a trace; the failure score is

$$F = 0.4\cdot 1_{\text{coll}} + 0.2\cdot 1_{\text{no goal}} + 0.2\cdot \sigma((2 - d_{\min})/0.5) + 0.1\cdot \min(n_{\text{replan}}/20, 1) + 0.1\cdot \text{dev}, \quad (1)$$

where $d_{\min}$ is the minimum clearance in metres and dev the normalised path deviation. A flight *fails* if it collides, leaves the geofence or times out. Analogous to DiffScene's safety, functionality and constraint objectives, F mixes safety (collision, clearance) and mission (goal, replanning) terms, while V plays the role of the constraint. Given a budget of B simulated flights, the red team seeks a set X of valid scenarios maximising the number of *distinct* failures,

$$\max_{|X| = B} |D(X)|, \quad D(X) = \{\, x \text{ in } X : \text{fail}(x), \text{ and } \|z(x) - z(x')\| \geq 1 \text{ for every earlier failure } x' \,\}, \quad (2)$$

where $z(\cdot)$ is the normalised scenario encoding, so that near-duplicates of an earlier failure do not count.

4 Method

4.1 Simulator, Planner and Real-Data Prior

The 2D simulator (60×60 m, 5 m geofence, dt = 0.05 s) models a UAV that tracks the planner's path with an acceleration limit of 5 m/s^2 and a turn-rate limit of 1.5 rad/s. Wind acts as a force with optional 3 s gusts. The planner perceives obstacles only within its sensing range, 0.2 s late and with Gaussian position noise, and replans every 3 s, on a new detection or when its path is blocked. Figure 2 shows a recorded failure.

Real-data prior. The procedural generator that produces the 10,000 training scenarios draws moving-obstacle speeds from 149,000 one-second track segments of the Stanford Drone Dataset (Robicquet et al. 2016), obtained via OpenTraj: pedestrians move at a median 1.2 m/s and cyclists at 3.5 m/s. Wind and gusts come from 8,753 hourly NOAA ISD observations at San Francisco airport in 2023 (NOAA NCEI 2023): median wind 4.1 m/s, gusts in 13% of hours.

Fairness rule. In an early version, search methods learned to spawn obstacles directly in front of the UAV, producing crashes no planner could avoid. We reject any scenario in which an obstacle appears closer to the UAV's nominal route position than its reaction distance

$$r = v\tau + v^2/(2a) + |v_o|(\tau + v/a) + 1 \text{ m}, \quad (3)$$

with UAV speed v, sensing delay $\tau = 0.2$ s, deceleration a = 5 m/s^2 and obstacle speed v_o. The rule cut unavoidable crashes from 18% to 6.5% of our method's failures and from 41% to 18% of CMA-ES's; the remainder arise when the UAV drifts off its nominal route and are flagged in the analysis.

4.2 Conditional Diffusion Generator

Scenario vectors are normalised to [−1, 1] and modelled with a DDPM (T = 200, cosine schedule). The denoiser is a residual MLP (width 512, 4 blocks, 3.4M parameters) with FiLM conditioning on the time step and a condition vector c = (C(x), F): seven coarse descriptors C (cruise speed, obstacle count, share of moving obstacles, wind magnitude and direction relative to travel, sensing range) and the difficulty F. Conditions are randomly masked during training so that any subset can be specified at sampling time. The model is trained with the standard objective

$$L = E_{x,\, \varepsilon,\, t} \|\varepsilon - \varepsilon_\theta(x_t, t, c)\|^2 \quad (4)$$

and sampled with classifier-free guidance, $\hat{\varepsilon} = \varepsilon_u + w(\varepsilon_c - \varepsilon_u)$, where ε_u uses the null condition. Asking for F = 1 with w > 1 steers samples toward difficulty. DiffScene instead adds gradients of a learned adversarial objective at each

denoising step; our variant needs no differentiable surrogate of the planner, which is a non-differentiable sampling-based algorithm.

4.3 Closed-Loop Red-Teaming

Algorithm 1 spends the budget over R = 5 rounds. Failures with F ≥ 0.5 enter a buffer unless they lie within 0.5 of a buffered scenario; after each round the generator is fine-tuned on the seed data plus five copies of the buffer, so that the model's notion of "hard" moves toward the failures the planner actually exhibits.

Algorithm 1 Closed-loop diffusion red-teaming

Input: seed set $D_0 = \{(x_i, F_i)\}$, denoiser ε_θ, budget B, rounds R, guidance w, threshold τ_F

train ε_θ on D_0 with condition c = (C(x), F)
buffer $M \leftarrow$ empty set
for r = 1 … R **do**
  sample $x \sim p_\theta(x \mid F = 1)$ with CFG weight w; keep valid x
  fly each x in the simulator (B/R flights); compute F(x)
  add x with $F(x) \geq \tau_F$ to M unless a near-duplicate is already in M
  fine-tune ε_θ on D_0 plus 5 copies of M (800 steps)
end for
re-run each failure under 6 single-factor interventions (Eq. 5)

Output: failures with measured causes, passed to the LLM analyst

4.4 Measured Causes

Each failure x is re-flown with the same planner seed under six interventions T_k: no wind, full sensing range (25 m), no sensor noise, late obstacles present from t = 0, 30% slower cruise speed, and three fresh planner seeds (fixed if at least two succeed). Factor k is a *measured cause* if

$$\text{cause}_k(x) = \text{fail}(x) \text{ and not fail}(T_k(x)), \qquad (5)$$

and the set of causes defines the failure *type*; a failure that no single intervention prevents is *compound*.

4.5 Grounded LLM Analyst

For each failure we assemble an evidence record: scenario parameters, trace measurements (clearance, detection time and gap, replans, wind felt), counterfactual outcomes, and the three most similar past failures retrieved by k-nearest neighbours from the failure library together with their measured types. Gemini (gemini-3.1-flash-lite, free tier) must return JSON naming factors from a fixed vocabulary, each citing the evidence field that supports it. A script rejects citations of missing fields and of the retrieved cases; the simulator, never the LLM, decides the outcome.

5 Experiments

Setup. Six methods each fly 2,000 scenarios per run, over three seeds: random sampling of the procedural generator; CMA-ES maximising F directly over the 46 numbers (population 16, 4 restarts); the cVAE and the diffusion model asked for maximum difficulty (w = 2), each with and without feedback. The cVAE (latent 32, β = 0.3 with KL warm-up and free bits) was selected by a four-point β sweep after a first version suffered posterior collapse.

Metrics. Failures; *distinct* failures (Eq. 2); *coverage*, the number of occupied cells of a 384-cell grid over speed, obstacle count, moving share, wind and sensing range; failure *types* among each run's first 200 failures (fair across methods with different failure counts); and the number of flights needed to reach the 50th distinct failure.

Table 1: Method comparison, 2,000 flights per run, mean ± std over 3 seeds. Cover. = occupied cells of 384; Types = failure types among the first 200 failures; To 50th = flights until the 50th distinct failure.

Method	Failures	Distinct	Cover.	Types	To 50th
Random	215±15	214±15	112±2	28±2	461±51
CMA-ES	**743±109**	510±71	31±2	8±3	234±51
cVAE	262±4	216±2	84±5	**36±4**	374±48
cVAE + fb	295±9	209±8	80±3	35±4	375±51
Diffusion	445±19	445±20	**116±1**	**36±3**	**230±22**
Diff. + fb (ours)	553±22	**552±21**	109±2	35±4	**230±22**

5.1 Main Comparison

Table 1 and Figure 3 summarise the comparison. Diffusion with feedback finds the most distinct failures (552), 2.6× random sampling. CMA-ES finds the most *raw* failures (743), but 31% of them are near-duplicates and 88% belong to a single type (wind only, slow UAV in strong wind): it covers 31 grid cells against our 109 and finds 8 failure types in its first 200 failures against our 35. Its discovery curve also flattens late in the budget, while ours keeps rising (Fig. 3). This mirrors DiffScene's observation that search-based adversaries find many but narrow failures, while a learned prior keeps them diverse. Plain diffusion and random sampling cover slightly more cells than our method, which concentrates its budget where failures are; the cVAE matches us on failure types per 200 failures but finds 2.6× fewer distinct failures.

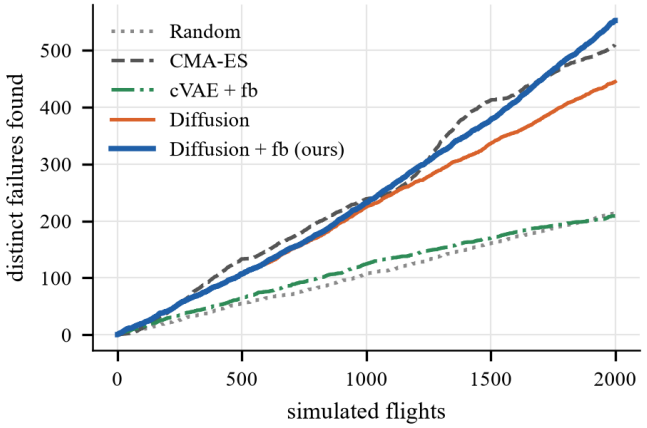


Figure 3: Cumulative distinct failures as the flight budget is spent (mean of 3 seeds). CMA-ES leads early, then flattens as it exhausts one failure mode.

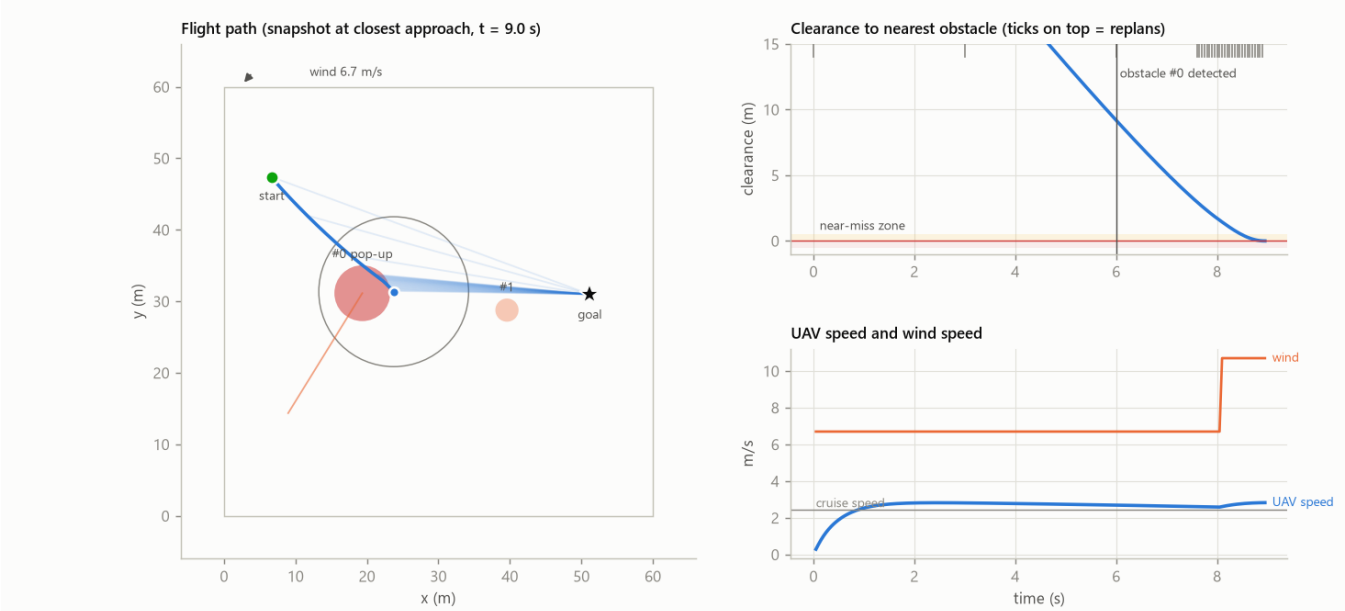


Figure 2: A failure from the random baseline, as recorded by the simulator. Left: flight path at the closest approach; the circle is the sensing range and obstacle #0 appears late (pop-up), detected at t = 6 s. Right: clearance and speed over time. A 4 m/s gust at t = 8 s pushes the UAV into the obstacle it is avoiding. Re-flying the same scenario with wind removed (and nothing else changed) gives a safe flight, so wind is a *measured* cause; no other single intervention prevents the crash.

5.2 Why Diffusion Rather than a VAE?

DiffScene motivated diffusion over VAE priors by realism in the tail of the distribution. Table 2 reproduces this argument in the UAV domain. Without conditions, 87% of diffusion samples pass the validity rules against 50% for the best cVAE, and diffusion's condition features are 2.1× closer (Wasserstein distance normalised by the data's standard deviation) to the real training distribution. Lowering the cVAE's KL weight sharpens its samples but makes validity worse (22–39%), so no setting achieves both. Diffusion is also steerable: asking for higher difficulty raises its failure rate monotonically to 35% at w = 4, while the cVAE never exceeds 13% (Fig. 4a). Generated scenarios are novel, not memorised: their median distance to the nearest training scenario (1.04) is close to that of fresh random scenarios (1.10).

Table 2: Generator quality. Valid = share of unconditional samples passing the validity rules; W-dist. = mean normalised Wasserstein distance of condition features to the training data (lower is more realistic); Fail@F=1 = failure rate when asked for maximum difficulty.

Generator	Valid	W-dist. ↓	Fail@F=1
cVAE β = 0.02	23%	0.31	–
cVAE β = 0.05	32%	0.31	–
cVAE β = 0.1	39%	0.36	–
cVAE β = 0.3 (used)	50%	0.36	13%
Diffusion (w = 2 / 4)	**87%**	**0.17**	**23% / 35%**

5.3 Does Feedback Help?

For diffusion, yes: over five rounds of fine-tuning on its own failures the failure rate rises from 21% to 36%, while without feedback it stays near 22% (Fig. 4b). Validity also rises (76% to 84%) because buffered failures are valid by construction. The cVAE barely benefits (13% to 15%): fine-tuning cannot fix a prior that produces mostly invalid samples.

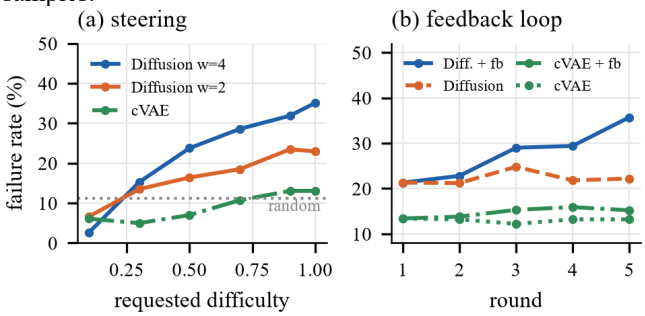


Figure 4: (a) Failure rate when the generator is asked for a given difficulty; the dotted line is random sampling. (b) Failure rate per round of the red-team loop (3 seeds).

5.4 Are the LLM's Explanations Faithful?

We explained 50 failures four times each with different evidence (200 explanations) and scored every named factor against the measured causes (Table 3, Fig. 5). From the trace alone the LLM names only 34% of the true causes and 44% of its explanations blame a factor the re-runs rule out. Adding retrieved similar cases, a form of retrieval-augmented generation, does not help: recall stays at 36% and contradictions rise to 60%, because the model tends to copy the failure type of its neighbours. With measured counterfactuals, recall reaches 92–96% with at most 2% contradictions, and no explanation cited missing evidence. Because the counterfactual variants are scored against evidence they were shown, their scores measure faithful *use* of evidence; the independent finding is the first two rows: causes should be measured by the simulator and only interpreted by the LLM.

Table 3: LLM analyst ablation (50 failures × 4 evidence variants). Contradicted = share of explanations naming a cause that the counterfactual re-runs rule out.

Evidence given	Precision	Recall	Contradicted
Trace only	63%	34%	44%
+ similar cases (retrieval)	59%	36%	60%
+ counterfactuals	**100%**	92%	**0%**
All evidence	98%	**96%**	2%

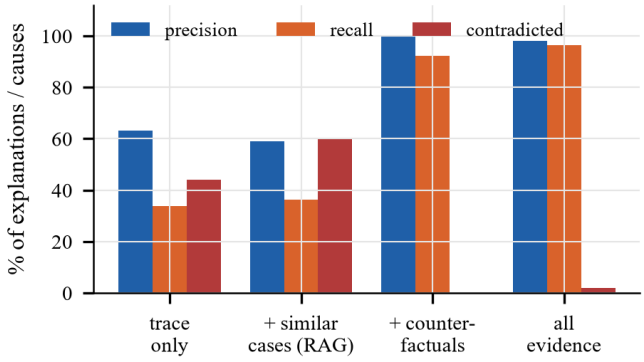


Figure 5: Faithfulness of the LLM's explanations by evidence given.

5.5 Are the Failures Real?

A 2D simulator invites the question whether failures are numerical artefacts. We re-flew 500 failures from random sampling and 500 from our method at the original step and at 2× and 4× finer steps, each with three fresh planner seeds. 78% of our failures still fail at the original step (the rest depend on RRT's random sampling) and 76% at the 4× finer step, the same as random-sampling failures (78% / 76%). On 2,000 random scenarios RRT* fails 12.4% (RRT 12.2%), and 77% of the failures found against RRT also break RRT*, with the same measured causes. The red team thus finds planner-level weaknesses rather than simulator artefacts or quirks of one planner.

5.6 Closing the Loop: Harden and Re-attack

The measured causes point to four fixes around the unmodified planner: wind feed-forward in the controller (wind is a cause in 79% of baseline failures), a speed governor tied to sensing range ("too fast to react", 46%), best-of-three planning (planner randomness, 25%) and a two-second swept footprint for moving obstacles. Replaying all 8,650 failures found so far with the same planner seeds, the hardened system survives 79% (Table 4). Compound failures are hardest to fix (42% survived), and failures found by CMA-ES are the most persistent (65%). On 3,000 ordinary scenarios the failure rate falls from 12.0% to 4.7% at the cost of 6% longer flights and 61 new failures. Re-attacking the hardened system, our red team finds 176 distinct failures per 2,000 flights against 84 for random sampling, and late detection rises to the second most common cause, pointing to the next fix.

Table 4: Hardening derived from measured causes: regression on past failures, side effects on 3,000 ordinary scenarios, and a re-attack (2 seeds).

Measure	Original	Hardened
Past failures survived (of 8,650)	0%	**79%**
Failure rate, ordinary scenarios	12.0%	**4.7%**
Near-failure rate	4.0%	0.9%
Mean flight time to goal	12.0 s	12.7 s
Re-attack: random, distinct / 2k	220	84
Re-attack: ours, distinct / 2k	552	176

6 Limitations

- **Simplified world.** 2D kinematics with acceleration and turn limits, a wind force and delayed noisy sensing, but no aerodynamics or 3D geometry; results do not transfer directly to real quadrotors.
- **Partial grounding.** Obstacle speeds and wind follow real data; obstacle layouts, sensing parameters and the UAV model are synthetic, and the wind comes from one airport.
- **Residual unavoidable failures.** The fairness rule uses the nominal route; 6.5% of our failures (18% of CMA-ES's) still involve an obstacle appearing within braking time after the UAV drifted.
- **Metric choices.** The duplicate threshold, coverage grid and type definition are ours; with raw failure counts CMA-ES ranks first. Only two failure types were found exclusively by our method.
- **Evaluation scale.** Three seeds per method; the LLM was scored automatically on 50 cases with one model and no human rating.

7 Conclusion

We transferred DiffScene's central idea, a diffusion model as a realistic prior for safety-critical scenario generation, from driving to UAV path planning, and closed the loop from generation to explanation to repair. A closed-loop conditional diffusion model finds the most distinct failures of an unmodified RRT stack and spreads them over far more of the scenario space than direct search, while remaining valid and realistic where a tuned cVAE does not. Measured counterfactual causes, not retrieval, make LLM explanations faithful, and they translate into targeted fixes that remove most known failures. The hardened planner still fails, and the red team still finds those failures twice as fast as random testing, which is the point of red-teaming as a continuing process. Natural next steps are 3D dynamics, wind fields learned from more stations, and guidance with a learned failure critic as in DiffScene.

Code and Data Availability

All code, trained-model recipes, results and the demo video are at https://github.com/Snehil06/cs787-uav-redteaming. Appendix A lists where each part of this paper lives in the repository. The planner is PythonRobotics (https://github.com/AtsushiSakai/PythonRobotics); obstacle speeds come from the Stanford Drone Dataset via OpenTraj (https://github.com/crowdbotp/OpenTraj); wind comes from NOAA ISD (https://www.ncei.noaa.gov/products/land-based-station/integrated-surface-database); the LLM is the Gemini API (https://ai.google.dev).

References

Corso, A.; et al. 2021. A Survey of Algorithms for Black-Box Safety Validation of Cyber-Physical Systems. *Journal of Artificial Intelligence Research* 72: 377–428.

Hansen, N. 2016. The CMA Evolution Strategy: A Tutorial. arXiv:1604.00772.

Ho, J.; Jain, A.; and Abbeel, P. 2020. Denoising Diffusion Probabilistic Models. In *NeurIPS*.

Ho, J.; and Salimans, T. 2021. Classifier-Free Diffusion Guidance. In *NeurIPS Workshop on Deep Generative Models*.

Karaman, S.; and Frazzoli, E. 2011. Sampling-Based Algorithms for Optimal Motion Planning. *IJRR* 30(7): 846–894.

LaValle, S. M. 1998. Rapidly-Exploring Random Trees: A New Tool for Path Planning. Tech. Rep. 98-11, Iowa State University.

NOAA NCEI. 2023. Integrated Surface Database, station 72494023234 (San Francisco Intl.).

Rempe, D.; et al. 2022. Generating Useful Accident-Prone Driving Scenarios via a Learned Traffic Prior. In *CVPR*.

Robicquet, A.; et al. 2016. Learning Social Etiquette: Human Trajectory Understanding in Crowded Scenes. In *ECCV*.

Sakai, A.; et al. 2018. PythonRobotics: A Python Code Collection of Robotics Algorithms. arXiv:1808.10703.

Sohn, K.; Lee, H.; and Yan, X. 2015. Learning Structured Output Representation Using Deep Conditional Generative Models. In *NeurIPS*.

Wang, J.; et al. 2021. AdvSim: Generating Safety-Critical Scenarios for Self-Driving Vehicles. In *CVPR*.

Xu, C.; Petiushko, A.; Zhao, D.; and Li, B. 2025. DiffScene: Diffusion-Based Safety-Critical Scenario Generation for Autonomous Vehicles. In *AAAI* 39(8): 8797–8805.

Xu, C.; Zhao, D.; Sangiovanni-Vincentelli, A.; and Li, B. 2023. DiffScene: Diffusion-Based Safety-Critical Scenario Generation for Autonomous Vehicles. In *AdvML-Frontiers Workshop at ICML*.

Appendix A Repository Guide

The repository has three folders. **report/** holds this paper. **results/** holds one folder per experiment; inside each, *1_runs* has flight pictures and videos, *2_results* the charts and CSV tables, and *3_crash_reasons* the measured causes and explanations of individual failures. **code/** holds the full source, laid out as in Figure 1. A quick tour: read the top-level README, open the end-to-end case study (one real crash followed from generation to fix and re-attack), then the results folder of any section of interest. All paths below are links.

Table A1: Where to find each part of the paper in the repository.

Path (link)	What it contains
README.md	Start here: overview, headline numbers, map from paper sections to results
report/report.pdf	This paper
results/demo.mp4	2.5-minute video: example flights and the main results
results/08_case_study/case_study.pdf	**One real case end to end**: crash created, causes measured, LLM explanation, fix, re-flown safely, attacked again (also as a video)
results/README.md	Index of all results, stage by stage
results/01_random_baseline	Random baseline and the real-data scenario prior (Sec. 4.1, Fig. 2)
results/02_generators	Diffusion vs. cVAE: validity, realism, steering (Sec. 5.2, Table 2)
results/03_red_team_loop	Effect of feedback fine-tuning, per round (Sec. 5.3)
results/04_llm_analyst	LLM ablation and every explanation with its score (Sec. 5.4, Table 3)
results/05_comparison	Main comparison of all methods (Sec. 5.1, Table 1, Fig. 3)
results/06_fidelity_checks	Finer time steps and RRT* transfer (Sec. 5.5)
results/07_harden_and_reattack	Hardening, side effects, re-attack (Sec. 5.6, Table 4)
code/README.md	Setup and the command for every stage
code/scenarios	Scenario vector, validity and fairness rules, real-data statistics
code/sim/simulator.py	2D UAV simulator (dynamics, wind, delayed sensing, replanning)
code/planners	Unmodified PythonRobotics RRT / RRT* and its adapter
code/generators/diffusion.py	Conditional DDPM with classifier-free guidance (Sec. 4.2)
code/generators	Also the cVAE and CMA-ES baselines
code/loop/scoring.py	Failure score F (Eq. 1)
code/loop/redteam.py	Closed red-teaming loop (Sec. 4.3, Algorithm 1)
code/loop/metrics.py	Distinct failures, coverage, failure types
code/analyst	Counterfactual re-runs, evidence records, Gemini analyst, grounding check (Sec. 4.4-4.5)
code/experiments/run_all.py	Runs the whole pipeline in order (about 2.5 h)